

CSE422: Artificial Intelligence

LAB MID EXAM

TOTAL MARKS: 8

DURATION: 60 MINUTES

Note: You may create arbitrary input and output cases to test your designed algorithms.

Problem 1: A* Search [3.5 Marks]

Task: Implement a function `a_star(graph, heuristics, start, goal)` to find the optimal path from a starting node to a goal node using the A* Search algorithm.

Inputs:

- `graph`: A dictionary where keys are node characters (e.g., 'A') and values are lists of tuples representing neighbors and step costs (e.g., ('B', 2)).
- `heuristics`: A dictionary mapping each node character to its integer heuristic value.
- `start, goal`: Single characters representing the start and goal nodes.

Output: A single list of characters representing the optimal path. Do not return the cost.

Sample Input:

```
graph = {
    'S': [('A', 4), ('B', 2)],
    'B': [('A', 1), ('C', 5)],
    'A': [('C', 2)],
    'C': []
}
heuristics = {'S': 6, 'A': 2, 'B': 4, 'C': 0}
```

Sample Output:

```
['S', 'B', 'A', 'C']
```

Problem 2: Genetic Algorithm [4.5 Marks]

Scenario: You are a football manager tasked with assembling an optimal fantasy squad. Given a pool of players, each with a specific match rating and transfer cost, your goal is to maximize the team's total rating. A chromosome is represented as a binary string where '1' means a player is selected for the squad, and '0' means they are left out.

Task: You must implement the following four helper functions and combine them inside a main `generate_evolution` function to simulate exactly 10 generations of a Genetic Algorithm.

1. `fitness(chromosome, ratings, costs, budget)`: Calculates the fitness of a squad based on selected players. If the total transfer cost exceeds the budget, a penalty is applied:
$$Fitness = \max(0, Total\ Rating - 5 \times (Total\ Cost - Budget))$$

2. `select_parents(population, fitness_scores)`: Performs truncation selection. Sorts the population based on fitness in descending order and returns the fittest 3/4ths of the population to act as the mating pool.
3. `crossover(parent1, parent2)`: Performs a single-point crossover at a randomly chosen index to create two new offspring.
4. `mutate(chromosome)`: Randomly flips exactly one bit ($0 \rightarrow 1$ or $1 \rightarrow 0$) in the given chromosome.
5. `generate_evolution(population, ratings, costs, budget)`: Uses the above 4 functions sequentially. For exactly 10 iterations (generations): evaluate the population, apply truncation selection to get the fittest 3/4ths, randomly select pairs from this mating pool to perform crossover until you have enough offspring to replace the old population, and mutate the offspring. Return the final population.

Inputs for `generate_evolution`:

- `population`: A list of binary strings (e.g., ['1110', '0110', '1000', '0011']).
- `ratings, costs`: Lists of integers representing the match rating and transfer cost of each player.
- `budget`: An integer representing the maximum budget before penalties apply.